TV5: LOCKING, BYZANTINE FAULTS AND DETERMINISTIC LOGS

Implementation notes and experimental results | Python 3.10+ | Tendermint-style BFT | n = 3f + 1

1. Scope and safety objective

TV5 implements the safety boundary between untrusted consensus messages and the validator state machine. VoteTracker validates and de-duplicates votes; LockManager enforces local signing, PRECOMMIT prerequisites, locking and PoLC-based lock changes; ByzantineAttackSimulator creates bounded faults; and DeterministicLogger produces byte-stable evidence. The core invariant is that no two different blocks can be finalized at the same height by honest nodes.

2. Seven voting and locking rules

Rule	Enforced condition	Implementation
1	At most one PREVOTE per validator and (H, R).	Local signing slot keyed by height, round and phase.
2	At most one PRECOMMIT per validator and (H, R).	PRECOMMIT uses a separate phase slot.
3	Non-NIL PRECOMMIT requires a matching PREVOTE quorum.	can_precommit() checks a stored PoLC certificate.
4	2f + 1 PREVOTE(B), B != NIL, locks B at round R.	observe_quorum() validates and records the lock.
5	A different block requires locked_round < polka_round < proposal_round.	can_prevote() rejects stale or same-round PoLC data.
6	Reject wrong voter, height, round, phase, hash, chain or signature domain.	Optional crypto verification uses VOTE:<chain_id>.
7	Duplicates count once; conflicting duplicates are equivocation.	First valid vote remains; the pair is retained as evidence.

3. Quorum and PoLC validation

Votes are indexed by (height, round, phase) and tallied by block hash. A quorum result contains only distinct validators supporting the same block. A PoLC certificate is accepted only if all votes agree on height, round, PREVOTE phase and block hash, and its voter set is unique. Locks reset at a new height; the local signing history remains auditable.

```
Safety-critical ordering: locked_round < polka_round < proposal_round
```

4. Byzantine attack simulator

The simulator refuses configurations with more than f = floor((n - 1) / 3) faulty validators. All recipient sets are sorted before attacks are generated, so identical inputs produce identical deliveries.

Attack	Behavior	Expected defense
Equivocation	Signs block A for one network half and block B for the other.	Only one vote per validator is counted; conflict is evidence.
Invalid state	Proposes a deterministic incorrect state commitment.	Execution/ledger validation rejects the proposal.
Replay	Repeats an old packet or vote without mutation.	Vote and transaction de-duplication stay idempotent.
Silence	Withholds all outbound messages.	Timeout and round change preserve liveness.

5. Deterministic event logging

The logger never reads wall-clock time. Simulated timestamps are normalized with decimal round-half-even; structured details use compact sorted JSON; embedded line breaks are escaped; and files use UTF-8, LF and one final newline. These rules remove locale, dictionary-order, newline and floating-display variance.

6. End-to-end verification

Test	Configuration	Assertion	Result
T7	4 validators, 1 equivocator, seed 42	All observer views finalize one common block; no fork.	**PASS**
T8	Two executions of the same T7 scenario	Logs, finalized blocks and final state hash are identical.	**PASS**
Unit + E2E	34 repository tests	Rules, attacks, logger and summary artifacts.	**34/34**

Reproducibility evidence

```
Log SHA-256 (both runs)
3c5c2ac64672dc7a25746c1d5220d10cbe34f18277423def81a02bd2b9560afa

Final state hash
654698a006e8cdee6c249d932aa088ddbbc1cad4098284e52fb241b61f3b2f34
```

7. Integration status

The TV5 module, tests, logs and result summary are complete and independent. Full-stack revalidation with production Ed25519, execution state, network and consensus FSM is scheduled after the TV1-TV4 branches are merged. This avoids copying another member's implementation into the TV5 pull request and preserves clean ownership boundaries.

Artifacts: tests/e2e/test_tv5_byzantine_determinism.py | scripts/verify_tv5_determinism.py | logs/t8/results.json